

第3回：Webアプリへの組み込み

ブラウザからFastAPIを経由してLLMを呼ぶ

今日のゴールと流れ

質問がどこで処理されるかを追いかける。

時間	内容
10分	前回のAPI呼び出しの復習
20分	画面とFastAPIの役割
20分	POST・JSON・入力検証
30分	fetchの実装と動作確認
10分	エラー確認・提出

通信の2段階

1. ブラウザが自分のFastAPIに質問を送る
2. FastAPIが外部のLLM APIに質問を送る
3. LLMの結果をFastAPIが受け取る
4. FastAPIがJSONを返し、ブラウザが表示する

ブラウザへ送るのは回答です。APIキーを送る必要はありません。

既存コードと今回の追加

前回から使う	今回加える
<code>settings.py</code>	<code>main.py</code> : 自作API
<code>llm.py</code> の <code>ask_plain()</code>	<code>static/index.html</code> : 入力画面
LLM APIの呼び出し	<code>static/app.js</code> : 送信と表示
入出力の辞書	<code>static/style.css</code> : 見た目

`session03` のコードには、検索やDBの機能はまだありません。

質問のJSON

ブラウザからFastAPIへ送る例です。

```
{  
  "question": "FastAPIとは？",  
  "mode": "plain",  
  "search_text": "",  
  "top_k": 3  
}
```

`search_text` と `top_k` は、後の検索用です。今回は使いません。

Pydanticによる入力の形

```
class Question(BaseModel):  
    question: str = Field(min_length=1, max_length=500)  
    mode: Literal['plain'] = 'plain'
```

型と長さをサーバーで検証します。

ブラウザの `maxlength` だけでは、別のプログラムからの入力を防げません。

空白だけの文字列は `strip()` の後にも確認します。

エンドポイント

```
@app.post('/api/ask')
def ask(request: Question):
    question = request.question.strip()
    result = ask_plain(question)
    return result
```

これは中心部分の抜粋です。実際のコードには入力検証、エラー処理、表示用の情報の追加があります。

FastAPIからLLMを呼ぶ

```
result = ask_plain(question)
```

第2回と同じ関数です。入力方法が `input()` から HTTPリクエストに変わっても、LLMとの通信は再利用できます。

通常の `def` のエンドポイントから、同期SDKを呼びます。
今回は独自の非同期処理やストリーミングは追加しません。

静的ファイルの配信

```
app.mount(  
    '/',  
    StaticFiles(directory=BASE_DIR / 'static', html=True),  
    name='static',  
)
```

API定義より後に書きます。公開対象は `static/` だけです。

アプリのディレクトリ全体を公開すると、設定やDBが漏れる危険があります。

実習1：サーバー起動

リポジトリのルートから実行します。

```
source .venv/bin/activate  
cd session03/exercise  
python main.py
```

8000番のポートを開きます。

画面は表示されますが、送信部分が穴埋めなのでまだ質問できません。

fetchによる送信

```
const response = await fetch('/api/ask', {  
  method: 'POST',  
  headers: {'Content-Type': 'application/json'},  
  body: JSON.stringify({question: question, mode: 'plain'})  
});  
const data = await response.json();
```

相対パスなので同じサーバーへ送ります。

JavaScriptのオブジェクトを `JSON.stringify()` でJSON文字列にします。

asyncとawait

`fetch()` の通信結果は、すぐには返りません。

- `async function` : 待ち時間のある処理を書く関数
- `await` : その処理の結果を待って次へ進む
- その間もブラウザ全体が固まるわけではない

`response.json()` も、本文の読み取りを待ちます。

実習2：POSTの穴埋め

1. `static/app.js` の `TODO` を探す
2. POST通信のコメントを外す
3. 仮の `throw new Error(...)` を削除する
4. ページを再読み込みして送信する
5. デモの固定文が表示されることを確認する

次に講師指定のAPIモードで同じ操作を確認します。

Networkタブで観察

開発者ツールのNetworkタブで `/api/ask` を選びます。

- Request Payload：送った質問と方式
- Status：200、422、502など
- Response：返ってきたJSON
- 時間：応答までの待ち時間

外部LLMへの通信はサーバー側なので、ここには直接表示されません。

待っている間の画面

```
button.disabled = true;  
statusText.textContent = '調べています...';
```

処理の最後には `finally` でボタンを元に戻します。
成功時も失敗時も、次の操作ができる必要があります。

送信前に古い回答と資料を消し、新しい質問の結果と混同しないようにします。

HTTPエラーの扱い

`fetch()` は、HTTP 400や500でも必ず例外になるわけではありません。

```
if (!response.ok) {  
  throw new Error('入力内容やAPI設定を確認してください。');  
}
```

通信自体の失敗と、サーバーが返したエラーを区別します。

安全な表示

```
document.getElementById('answer').textContent = data.answer;
```

LLMの出力も、ユーザー入力と同じように扱います。

`innerHTML` へ入れると、HTMLとして解釈してしまいます。

教材ではMarkdownの装飾を付けず、文字列として表示します。

実習3：異常な入力

`/docs` を開き、POST `/api/ask` を試します。

入力	期待する結果
空文字	422
空白だけ	422
501文字以上	422
<code>mode: "unknown"</code>	422

ブラウザの制約を通さなくても、サーバー側が止めます。

確認問題

1. `fetch('/api/ask')` は誰に送信している？
2. APIキーが必要なのは、どちらの通信？
3. `finally` がないと、失敗後の画面で何が起こり得る？
4. LLMが返したHTMLをそのまま実行してよい？

提出物と次回

提出： `static/app.js` と `worksheet.md` のGitHub URL。

ワークシートには、質問から回答表示までの処理を4段階で書きます。
HTTP 422を1例確認して記録します。

次回は、LLMへ授業資料を渡し、回答の根拠を確認します。